



ELISA
Enabling **Linux** in
Safety Applications

WORKSHOP

ELISA Workshop London, UK

June 9-11, 2026

Co-hosted with Canonical



Taming the Violation Tsunami

Static Code Analysis on Linux

Eli Gurvitz

L Langban Leima Singha

Mohamed Saad Abdelhameed

Vito Magnanimo

NVIDIA



The Violation Tsunami

Running Coverity on Linux 6.17 configured with defconfig for X86

Analysis summary report - MISRA:

```
-----  
Files analyzed           : 6816 Total  
      C                  : 6816  
  
Total LoC input to cov-analyze : 2354139  
Functions analyzed       : 163775  
Classes/structs analyzed : 36926  
Paths analyzed          : 10847562  
Time taken by analysis   : 00:54:56  
Defects/Coding rule violations found : 2,648,405  
Total
```

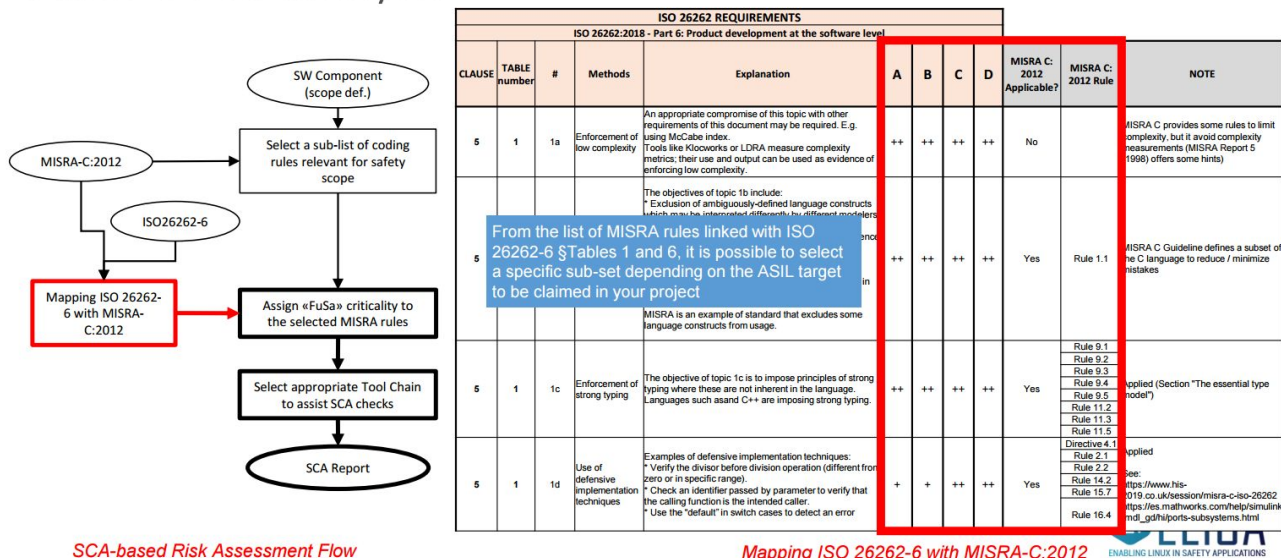
Analysis summary report - CERT-C:

```
-----  
Files analyzed           : 5716 Total  
      C                  : 5716  
  
Total LoC input to cov-analyze : 2354139  
Functions analyzed       : 163775  
Classes/structs analyzed : 36926  
Paths analyzed          : 12484250  
Time taken by analysis   : 00:50:40  
Defects/Coding rule violations found : 344,983 Total
```

Prior solutions – partial scanning (by ISO 26262)

Slide from a past ELISA presentation about Intel's Safety Concept for Safe Linux

Static code analysis



SCA-based Risk Assessment Flow

Mapping ISO 26262-6 with MISRA-C:2012

ENABLING LINUX IN SAFETY APPLICATIONS

Prior solutions – Risk assessment

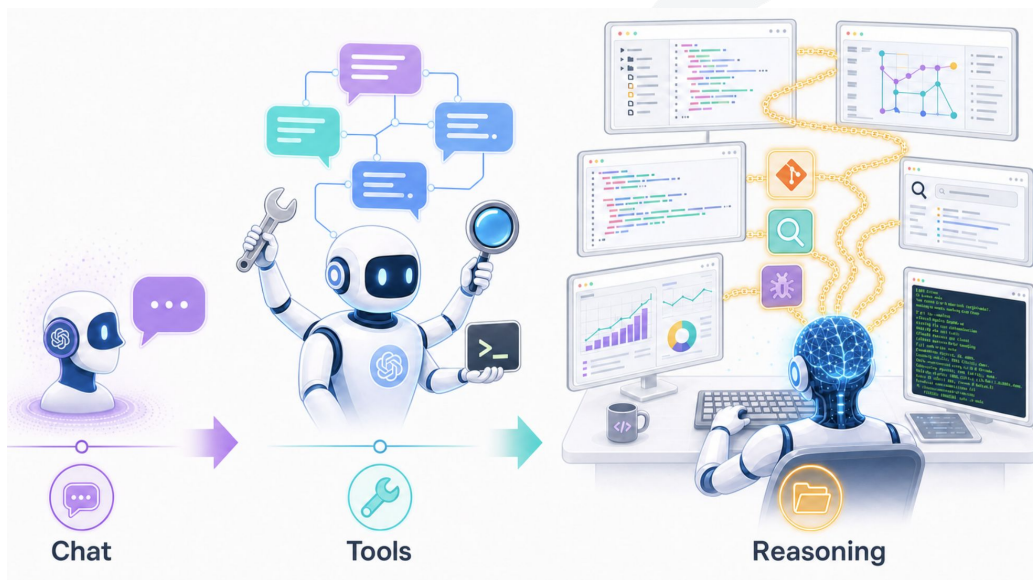
- No analysis of violations
- Use raw violations for Risk Assessment
 - Areas of code with more violations, or more critical violations, are more “risky”
- However, most of the violations aren’t problems at all and pose no risk
 - Using raw violations is meaningless

Prior solutions - additional solutions

- Select MISRA rules with few violations
 - Rules with many violations indicate a valid coding pattern
- Apply some MISRA rules to Safety-Critical parts of the code
- Claim that Static Code Analysis is not useful for pre-existing and complex SW

What changed?

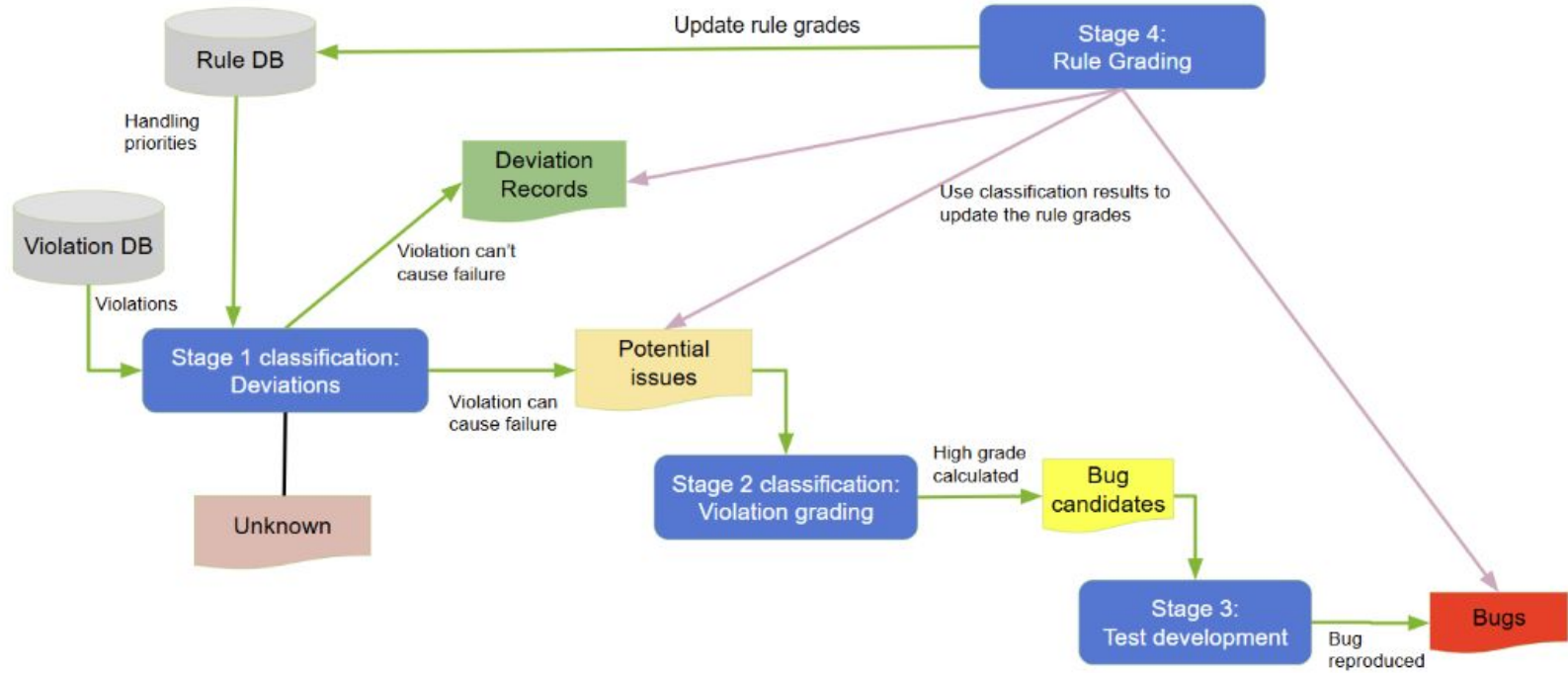
- Evolution from ChatGPT to reasoning models
 - From answering questions to reasoning models that use tools, hold a long chain of thought, can access source code repositories and have deep understanding of the Linux kernel



Note on data

- All examples, source code snippets and performance data are based on Linux kernel version 6.17 built for x86 defconfig
- Static code analysis was done by Coverity

NVIDIA's AI-based approach



Stage 1- Classification

Stage 1 uses AI to classify violations into three categories:

- **Deviated** – There's **proof** (in the code) that the violation can't cause a failure
- **Potential issue**
 - There's an indication that the violation can cause a failure; or
 - There's no proof that the violation can't cause a failure
- **Unknown** – Can't reach a conclusion
 - Usually due to gaps in the context provided to the LLM
 - Our goal is to have zero unknowns by:
 - Improving the prompt
 - Improving the tools that are used to gather the context

Deviation example 1 – Rule 17.7

“The value returned by a function having a non-void return type shall be used”

- Violation: arch/x86/kernel/apic/apic.c:2492
- Function: lapic_resume
- Ignored function return value: apic_read

Why is it ok to ignore the return value of this function? (explanation by Claude)

The `apic_read(APIC_ESR)` here is **not being called to inspect the error status**. It's being called purely for its **hardware side effect**: reading the ESR completes the latch-and-clear cycle. The purpose is to drain/discard any accumulated errors during resume, not to act on them ...

The return value is intentionally meaningless in this context. You can see the contrast at line 1409 and 1420, where the same pattern *does* capture the return value -- because there the code wants to compare error state before and after enabling the error vector.

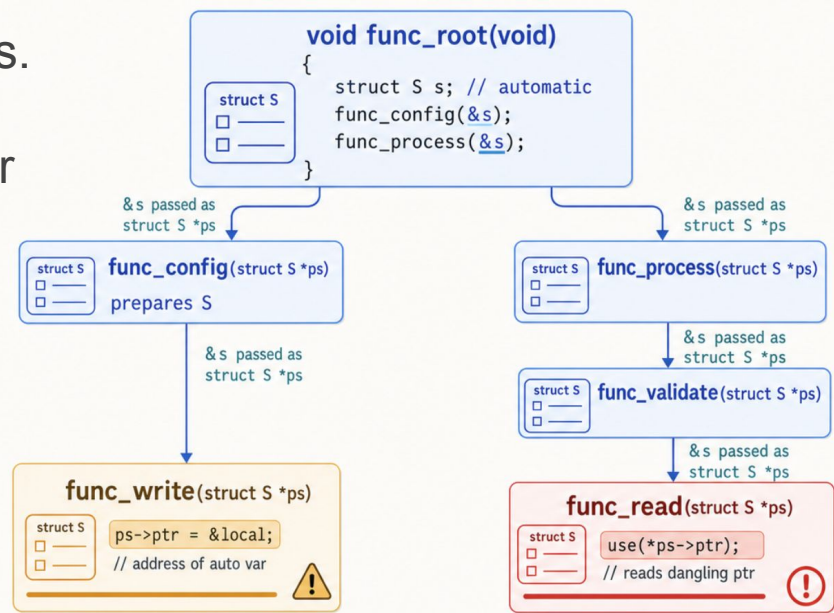
```
1418 |         if (maxlvt > 3)
1419 |             apic_write(APIC_ESR, 0);
1420 |         value = apic_read(APIC_ESR);
1421 |         if (value != oldvalue) {
1422 |             apic_pr_verbose("ESR value before enabling vector:
1423 |                 oldvalue, value);
1424 |         }
```

Deviation example 2 – Rule 18.6

“The address of an object with automatic storage shall not be copied to another object that persists after the first object has ceased to exist”

Surprisingly, rule 18.6 is violated 132 times.

If a full scan of the code reveals that `s->ptr` is not used in a downstream branch then this violation is “deviated”.



Stage 1 optimizations

- Deviation Permits (next slide)
- Reuse classification from previous versions
 - Limited usability because the analysis can be wrong for the new version
 - For example: in the new version some downstream function is using a variable in 18.6 violation

Stage 1 optimizations – Deviation Permits

- There are 7917 violations of rule 17.7 when using the `_printk` function
 - The return value of `_printk` is ignored
- This violation can't cause a failure
 - Unless someone designs a safety-related use case based on the success of `_printk` ...
- A Permit is created
- All 7917 violations are classified by **traditional string matching** and justified by the Permit

Stage 2 – Violation grading

- Violation grading is an optional stage
- Can be executed when resources or time are limited
- Aimed to focus attention to some Potential-issues above others
- Possible criteria:
 - Violations related to safety-critical use cases
 - Violations for which it is easier to find bugs

Stage 3 – Test development

- Use AI to create tests that demonstrate a **failure** based on a violation
- If a test is created, then this violation is considered a “**bug**”

Stage 4 – Rule grading

- This is an optional stage
- Rules are graded by tracking violations over multiple versions of Linux
- The grade indicates the applicability of a rule to Linux.
- For example:
 - If a rule has multiple violations but, over several versions, all the violations are “deviated” then the rule can be downgraded
- The grade can justify ignoring violations when resources are limited

Handling “deviated” violations

- Comply to the deviation procedure of each company
- For example, at NVIDIA, Deviation Records are created and reviewed
- Maybe the ELISA Tools WG can define a procedure for handling “deviated” violations

Addressing “potential-issues”

A potential-issue can be addressed in the following ways:

- Safety Analysis
- Specifications of AoUs
- Manual inspection
- Additional verification
- Fixing the violation

Addressing Potential-issue – example 1

- Violation:

- MISRA rule 17.7
- File: block/blk-core.c line: 386

```
382 static void blk_rq_timed_out_timer(struct timer_list *t)
383 {
384     struct request_queue *q = timer_container_of(q, t, timeout);
385
386     kblockd_schedule_work(&q->timeout_work);
387 }
```

- Analysis with Claude indicates that:

- “The critical link is the single call at line 386. The timer fires once, schedules one work item, and the work item is responsible for handling all currently expired requests AND re-arming the timer for any future ones. If that work item **does not run**, the entire timeout pipeline stalls.”

- **This** can happen if the work item is cancelled between the timer firing and the work item running.

- Mitigation: **AoUs (Assumption of Use)**: for example:

- Protect all block I/O with watchdogs; or
- Limit block I/O to the Init state and verify it succeeded before moving to Operational state

Addressing Potential-issue – example 2

- Violation:

- MISRA rule 12.2 – “Right hand operand of a shift operator shall lie between 0 and ... width -1”
- File: net/netlink/af_netlink.c line: 164

- The literal “1” is of type “int” – signed int

- The value $1 \ll 31$ is undefined

- The result is not guaranteed to be 0x80000000 and the compiler can optimize it in surprising ways

- Mitigations:

- Fix it:
 - Change the 1 to 1U or use `BIT(group - 1)` where: `#define BIT(nr) (1U << (nr))`
- Safety requirement + test

```
static inline u32 netlink_group_mask(u32 group)
{
    if (group > 32)
        return 0;
    return group ? 1 << (group - 1) : 0;
}
```

Addressing Potential-issue – example 3

- Violation:

- MISRA rule 9.1: “The value of an object with automatic storage duration shall not be read before it has been set”
- File: kernel/events/core.c line: 10036
- “sample” is used uninitialized

```
10025 static void perf_event_text_poke_output(struct perf_event *event, void *data)
10026 {
10027     struct perf_text_poke_event *text_poke_event = data;
10028     struct perf_output_handle handle;
10029     struct perf_sample_data sample;
10030     u64 padding = 0;
10031     int ret;
10032
10033     if (!perf_event_text_poke_match(event))
10034         return;
10035
10036     perf_event_header__init_id(&text_poke_event->event_id.header, &sample, event);
```

```
7722 void perf_event_header__init_id(struct perf_event_header *header,
7723                                struct perf_sample_data *data,
7724                                struct perf_event *event)
7725 {
7726     if (event->attr.sample_id_all) {
7727         header->size += event->id_header_size;
7728         __perf_event_header__init_id(data, event, event->attr.sample_type);
7729     }
7730 }
```

```
7694 static void __perf_event_header__init_id(struct perf_sample_data *data,
7695                                          struct perf_event *event,
7696                                          u64 sample_type)
7697 {
7698     data->type = event->attr.sample_type;
7699     data->sample_flags |= data->type & PERF_SAMPLE_ID_ALL;
```

Mitigations:

- **AoU**: Do not use the perf sub-system in safety-related use cases
- **Fix it**: Initialize the perf_sample_data struct

Is it feasible?

Analysis of MISRA rule 9.1 (“automatic variables must be set”):

- Context collection done by AI in multiple iterations, Deviation Record is produced
- Running on a VM with 8 vCPUs
- Number of violations: 584
- Total LLM queries: 1749
- Input tokens: 4,347,595
- Output tokens: 469,532
- **Elapsed time: 31 minutes**
- **Cost:** 

	Claude Sonnet 4	Nemotron-3-Super (NVIDIA NIM)
Price / 1M input	\$3.00	\$0.09
Price / 1M output	\$15.00	\$0.45
Input cost	\$13.04	\$0.39
Output cost	\$7.04	\$0.21
Total	\$20.08	\$0.60
Savings	—	~97% cheaper

This table was created by Perplexity.ai – data may be wrong or outdated

Is it feasible? with optimizations

Analysis of MISRA rule 8.5 (“An external object or function shall be declared once”):

- Optimization applied
- Creation of Deviation Record deferred
- Number of violations: 12010
- Total LLM queries: 3
- Input tokens: 3157
- Output tokens: 655
- **Elapsed time: 35 seconds**

Is it working? Yes

Zero-day security vulnerability found by this method

- Violation:
 - MISRA rule 17.7: “Return value of a function must be used”
 - File: crypto/scatterwalk.c
 - Line: 106 – 128
 - Function: memcpy_sglist
- Vulnerability:
 - Exposure of uninitialized kernel memory = leak of kernel memory to userspace

Skcipher vulnerability - details

```
void memcopy_sglist(struct scatterlist *dst, struct scatterlist *src, unsigned int nbytes)
{
    struct skcipher_walk walk = {};
    if (unlikely(nbytes == 0)) /* in case sg == NULL */
        return;
    walk.total = nbytes;
    scatterwalk_start(&walk.in, src);
    scatterwalk_start(&walk.out, dst);

    skcipher_walk_first(&walk, true);
    do {
        if (walk.src.virt.addr != walk.dst.virt.addr)
            memcopy(walk.dst.virt.addr, walk.src.virt.addr, walk.nbytes);
        skcipher_walk_done(&walk, 0);
    } while (walk.nbytes);
}
```

```
int skcipher_walk_first(struct skcipher_walk *walk, bool atomic)
{
    if (WARN_ON_ONCE(in_hardirq()))
        return -EDEADLK;

    walk->flags = atomic ? 0 : SKCIPHER_WALK_SLEEP;
    walk->buffer = NULL;
    if (unlikely(((unsigned long)walk->iv & walk->alignmask))) {
        int err = skcipher_copy_iv(walk);
        if (err) {
            return err;
        }
    }
    walk->page = NULL;
    return skcipher_walk_next(walk);
}
```

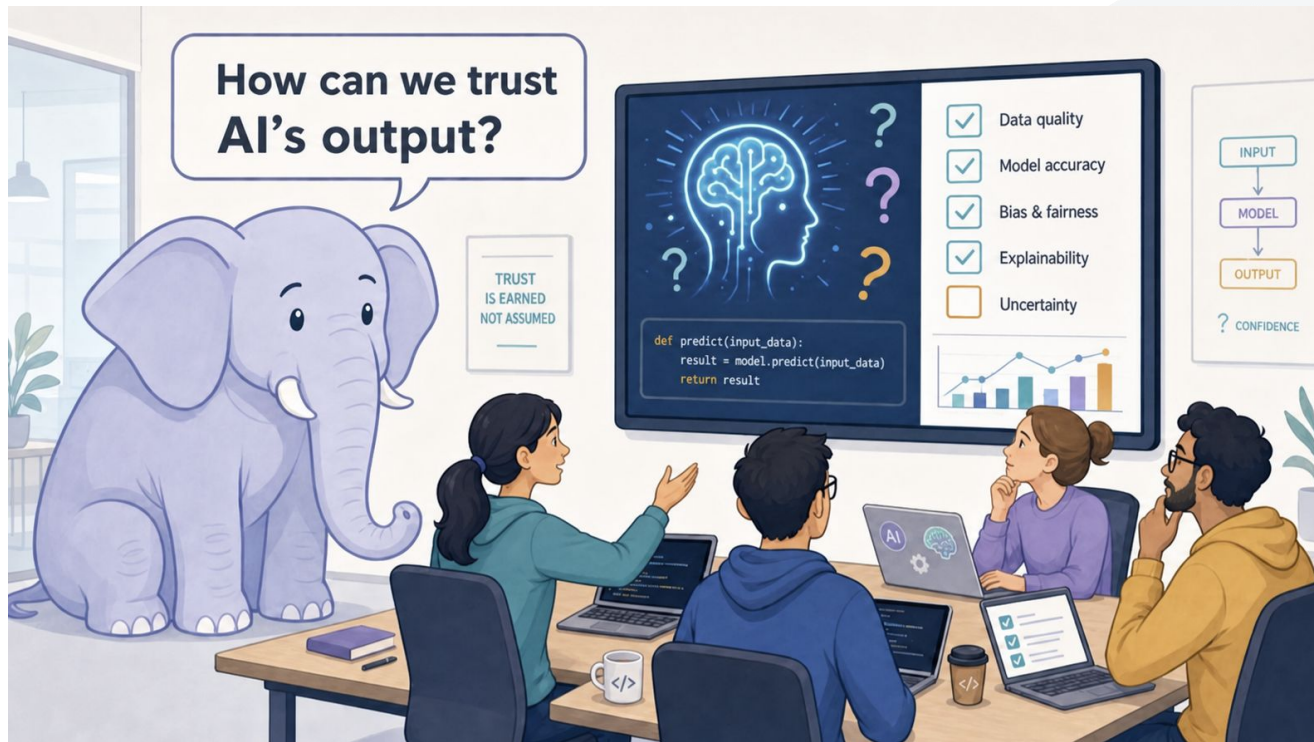
- skcipher_walk_first can fail ☐
- walk.src and walk.dst are not initialized and are both NULL ☐
- The do loop in memcopy_sglist is not done ☐
- The cipher operation, done on dst runs on uninitialized memory
- Incidentally, the **null cipher** uses memcopy_sglist ☐
- Leak of kernel memory to user-space

```
static int skcipher_copy_iv(struct skcipher_walk *walk)
{
    unsigned alignmask = walk->alignmask;
    unsigned ivsize = walk->ivsize;
    unsigned aligned_stride = ALIGN(walk->stride, alignmask + 1);
    unsigned size;
    u8 *iv;

    /* Min size for a buffer of stride + ivsize, aligned to alignmask */
    size = aligned_stride + ivsize + (alignmask & ~(crypto_tfm_ctx_alignment() - 1));
    walk->buffer = kmalloc(size, skcipher_walk_gfp(walk));
    if (!walk->buffer)
    {
        return -ENOMEM;
    }

    iv = PTR_ALIGN(walk->buffer, alignmask + 1) + aligned_stride;
    walk->iv = memcopy(iv, walk->iv, walk->ivsize);
    return 0;
}
```

Tool classification and qualification

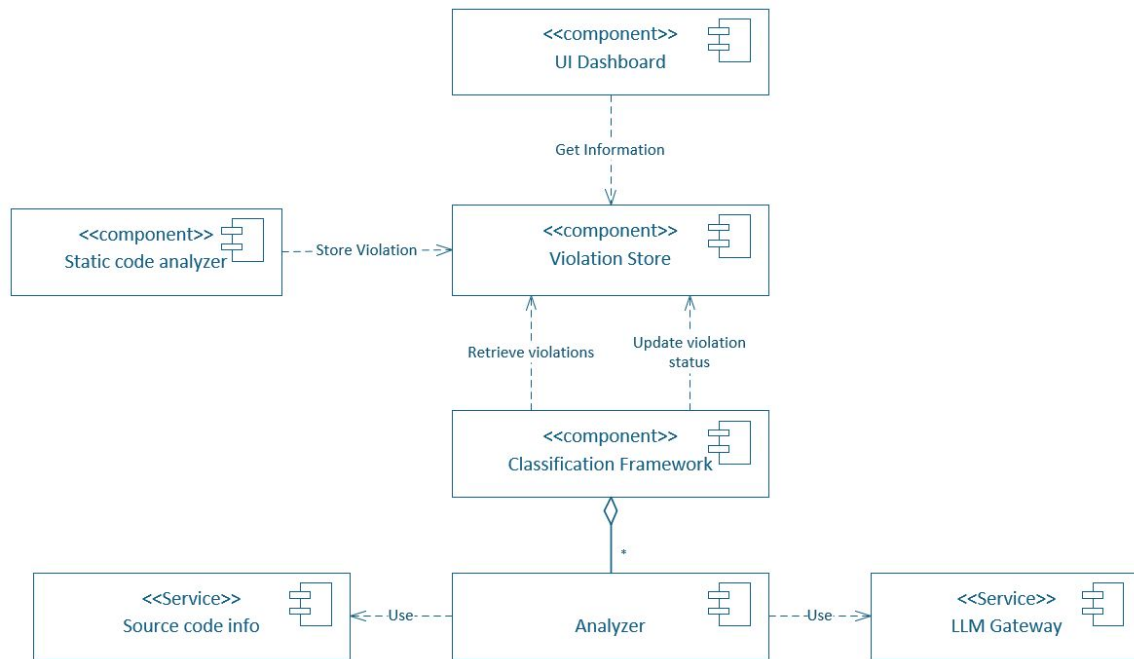


Program for the ELISA Tools WG

- The ELISA Tools WG can take on these topics:
 - Static code analysis
 - Use of AI in safety-related tools
 - Or more broadly: Tool Classification and Qualification for AI-based Tools
- Benefits of working on static code analysis:
 - No proprietary information
 - Saving time and effort for everybody
 - Many opportunities to contribute
 - The framework and adapters (see next slide)
 - The database
 - Analyzers / prompts
 - Reviews

System architecture

System architecture on which we can base the common work



Questions

Licensing of Workshop Results

All work created during the workshop is licensed under Creative Commons Attribution 4.0 International (CC-BY-4.0) [\[https://creativecommons.org/licenses/by/4.0/\]](https://creativecommons.org/licenses/by/4.0/) by default, or under another suitable open-source license, e.g., GPL-2.0 for kernel code contributions.

You are free to:

- Share — copy and redistribute the material in any medium or format
- Adapt — remix, transform, and build upon the material for any purpose, even commercially.

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms:

Attribution — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

No additional restrictions — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.



ELISA
Enabling **Linux** in
Safety Applications



WORKSHOP

